Mesh Triangulations for Biological Shape Analysis

Part 1: Intro to python and mesh triangulations

This tutorial introduces some core concepts in scientific programming and shape analysis using Python. We'll explore how to analyze and compare embryonic organ shapes using triangulated meshes. You will learn how to build meshes from scratch, analyze them for correctness, and register two shapes using Iterative Closest Point (ICP) alignment. The notebook is intended for early-stage graduate students with no serious coding experience.

---

0. Getting Started

0a. Pycharm

To begin working with Python code, we recommend using PyCharm, a free, user-friendly code editor. First, download PyCharm Community Edition and install it on your computer following the instructions here: `https://www.jetbrains.com/help/pycharm/installation-guide.html`. Also download the data folders from Box (`wildtype` and `bynGAL4_UASMyo1C`) and place them in the same place where you have the tutorial code cloned (see Figure 1A). When you open PyCharm for the first time, create a new project: choose the folder where this tutorial's code resides – same place as where the `wildtype` and `bynGAL4_UASMyo1C` data live (see Figure 1B).

Then configure a Python interpreter. Select or create an interpreter that uses Python 3.10 — this version is required for this tutorial. If no interpreter is available, click "Add Interpreter" and choose a Virtualenv, then select Python 3.10 as the base interpreter during setup (you may need to install it, see Figure 1C).

Default method: Once your project is open, open `organ_geometry.py`. At the top, some of the modules will be underlined in red since they are not installed. Hover over one and click `Install all missing packages` (see Figure 1D). Then close `organ_geometry.py` and open the Python Console using the top icon in the bottom left (see arrowhead in Figure 2). Now you can copy-paste snippets of code into the Python Console to run them (note the copy icon circled in Figure 2). If you see an error like `ModuleNotFoundError` for any reason while playing with the code, click the red light bulb or go to Settings > Python Interpreter to install the missing package. That's it — you're ready to start coding in Python!

Conda method for experienced students only: If you are experienced with python, feel free to use a Conda Environment and create a base interpreter from python 3.10. For Conda users, there is a .yml file included that will load up your virtual environment with what you need with:

```
$ conda env create -f environment.yml
```

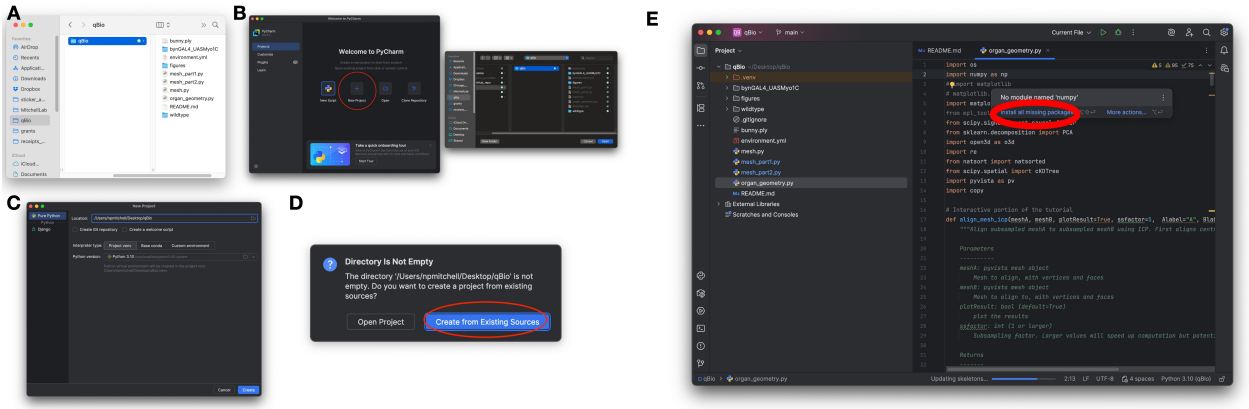

Figure 1: Setting up in PyCharm.

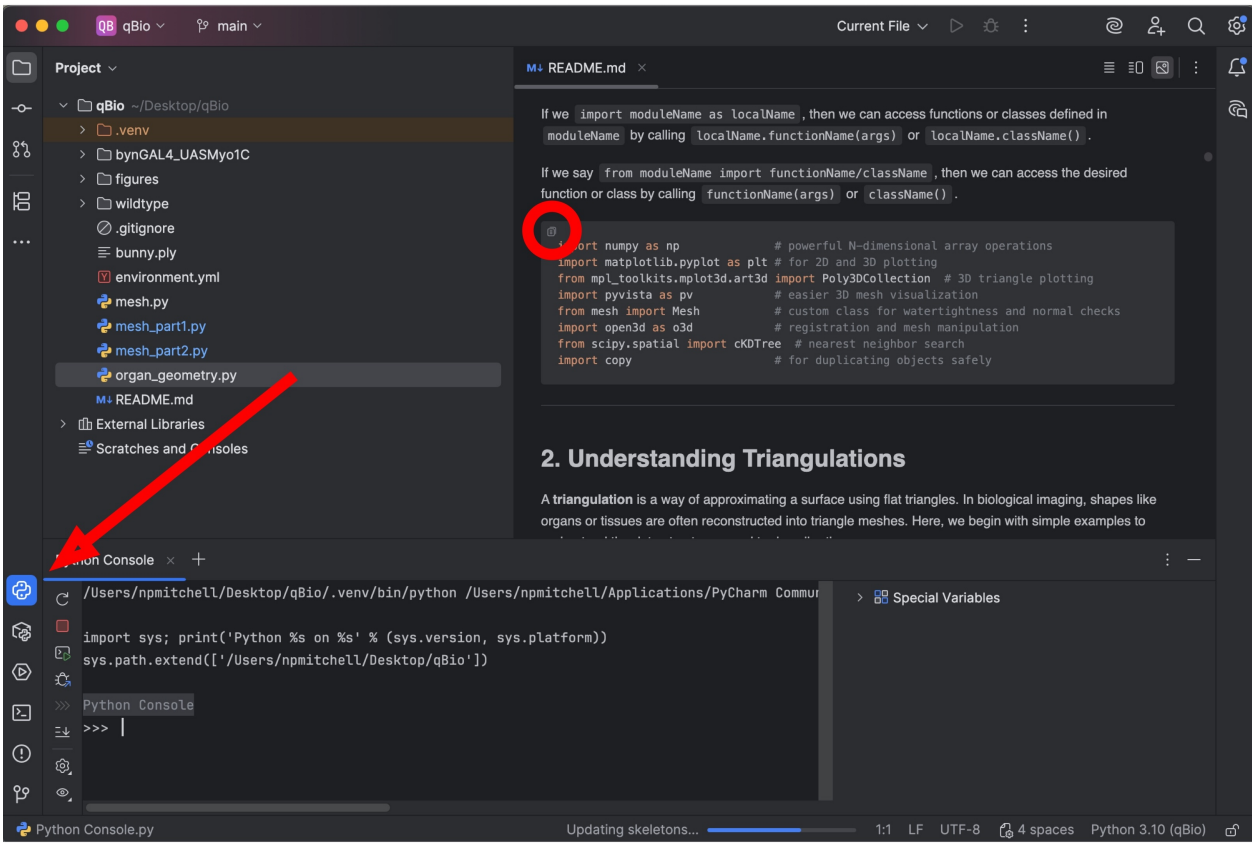

Figure 2: Running this module in PyCharm.

0a. Python

Python provides several core data structures that are widely used in both everyday scripting and advanced scientific workflows. Understanding their characteristics will help you choose the right tool for the job and write clearer, more efficient code.

We will work with the following fundamental data structures:

- **Lists**
  Lists are ordered, mutable collections of elements. They can hold elements of different types, and are typically used for sequential data or collections you plan to modify.
  Example: `mylist = [3.2, "gut", 5]`
- **Tuples**
  Tuples are like lists, but immutable. This means their contents cannot be changed after creation. Tuples are useful for representing fixed-size records or coordinates.
  Example: `coords = (1.0, 2.0, 3.0)`
- **Dictionaries**
  Dictionaries store key-value pairs. They are ideal for associating labels or identifiers with values, like mapping timepoints to filenames or experimental conditions to measurements.
  Example: `mydict = {"wt": "20240527", "oe": "20240626"}`
- **Arrays (from NumPy)**
  NumPy arrays are powerful containers for numerical data. Unlike lists, they support efficient mathematical operations and are used heavily in scientific computing, especially for vectors, matrices, and multidimensional datasets.
  Example: `arr = np.array([[1, 2], [3, 4]])`

Throughout this module, we'll use these data structures to work with 2D and 3D triangulated meshes, align shapes across space and time, and track shape variations of a model organ.

0c. The embryonic fly midgut

During development, tissues integrate mechanical and biochemical signals to drive organ-scale geometric transformations. Left-right symmetry breaking is a fundamental feature of organ morphogenesis that is crucial for organ function. While left-right asymmetric genetic patterning of the early embryonic body plan can trickle down to influence organ scale asymmetries, there is also a role for cell-intrinsic processes: cells themselves break left-right symmetry through cytoskeletal chirality. Remarkably, tuning the concentration of certain molecular motors such as members of the unconventional myosin 1 protein family can invert organ chirality across vertebrates and invertebrates alike, but we currently lack a physical understanding of the tissue-scale forces that drive organ-scale chirality. One direction in the Mitchell Lab aims to uncover the mechanical forces that drive organ asymmetry and relate them to cell-intrinsic activity of Myo1C. Here in this tutorial, we'll look at the chiral shape dynamics of the *Drosophila* midgut as a model system. We will ask: to what extent are the guts of wild-type and embryos with myosin 1C overexpression mirror images of one another?

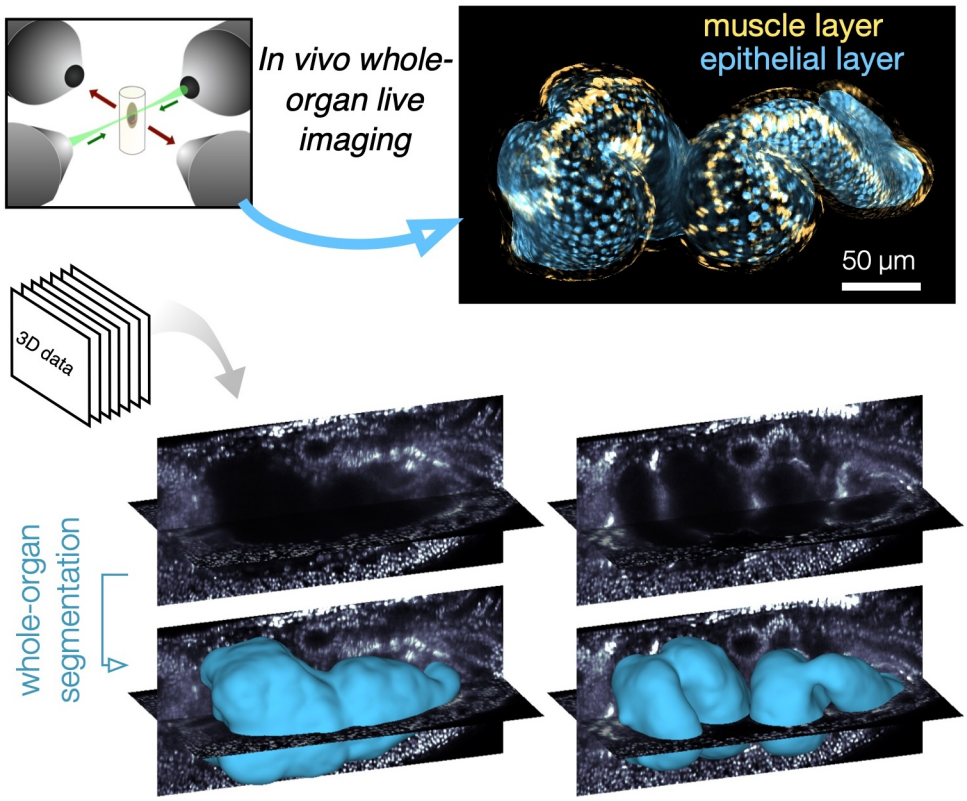

Figure 3: Multiview lightsheet imaging and computer vision tools enable analysis of chiral shape changes in gut morphogenesis.

The data you are working with was taken from live, multiview lightsheet microscopy imaging of the midgut across ~2.5 hrs of development. We used computer vision tools (Mitchell & Cislo, *Nature Methods* 2023) to extract the tissue surface over time. Briefly, we trained a two-stage 3D pixel classifier (iLastik autocontext workflow) to label the pixels of the endoderm and yolk as one probability channel and the muscle layer as another, then identified a 3D segmentation of the gut. The segmentation minimized an energy functional with surface tension, pressure, and attachment terms (see Active Contours Without Edges, see Márquez-Neila et al, *IEEE Trans Pattern Anal Mach Intell .* 2014 and Chan & Vese, *IEEE Transactions on Image Processing* 2001). Marching Cubes identified the segmentation's surface as a triangulation, which we then smoothed

with a Poisson disk reconstruction and Laplacian filters. Here you are presented with the resulting mesh triangulations.

---

1. Setup and Imports

Before we begin, we import the necessary Python libraries for 3D geometry processing, visualization, and numerical operations. Each of these libraries is a module. Note that you can write your own modules and import them, as long as they are placed somewhere that python can find them (ex in the current working directory or installed globally). As an example, we import the Mesh class from mesh.py, which is included in this tutorial.

If any of these other statements return an error, simply install the required module into your virtual environment in PyCharm (or on terminal if you have a Conda environment).

If we `import moduleName`, then we can access functions or classes defined in `moduleName` by calling `moduleName.functionName(args)` or `moduleName.className()`.

If we `import moduleName as localName`, then we can access functions or classes defined in `moduleName` by calling `localName.functionName(args)` or `localName.className()`.

If we say `from moduleName import functionName/className`, then we can access the desired function or class by calling `functionName(args)` or `className()`.

```
import numpy as np                # powerful N-dimensional array operations
import matplotlib.pyplot as plt # for 2D and 3D plotting
from mpl_toolkits.mplot3d.art3d import Poly3DCollection  # 3D triangle plotting
import pyvista as pv              # easier 3D mesh visualization
from mesh import Mesh             # custom class for watertightness and normal checks
import open3d as o3d              # registration and mesh manipulation
from scipy.spatial import cKDTree  # nearest neighbor search
import copy                       # for duplicating objects safely
```

---

2. Understanding Triangulations

A **triangulation** is a way of approximating a surface using flat triangles. In biological imaging, shapes like organs or tissues are often reconstructed into triangle meshes. Here, we begin with simple examples to understand the data structures used to describe them.

A mesh consists of: - a **list of vertices** (points in 2D or 3D space, stored as a NumPy array) - a **list of faces** (which connect 3 or more vertex indices to form polygons)

2a. An example triangulation: the Stanford Bunny

Let's first learn by example. We read in a PLY file of the Stanford Bunny, provided by the Stanford Computer Graphics Laboratory.

```
m = pv.read('bunny.ply')
print(m)
m.plot(show_edges=True)
```

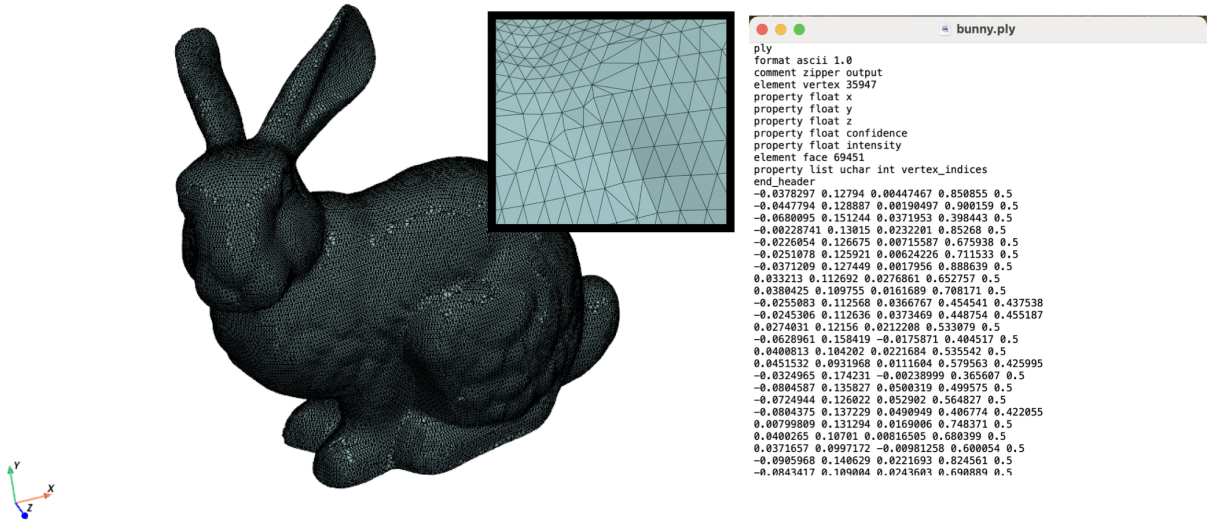

Figure 4: The Stanford bunny as a mesh triangulation, composed of vertices and faces.

How is this defined? It has vertices and faces. A face connects three vertices. Because the faces are triangles, this is a *triangulation*.

Open up `bunny.ply` in a text editor. See how it has vertices and faces defined (Figure 4).

2b. A Single Triangle

This is the simplest mesh, made of 3 points and 1 triangle. Here `x` and `y` are NumPy arrays and `faces` is a Python list of vertex index triplets.

```
x = np.array([0, 1, 0])          # x-coordinates of the vertices
y = np.array([0, 0, 1])          # y-coordinates
faces = [[0, 1, 2]]              # one triangle made from those three points

# Plotting in python: use matplotlib.pyplot
# Let's make lines that connect the vertices.
# Concept check: Why do we have to index into faces?
plt.figure()
plt.plot(x[faces[0]], y[faces[0]], '-')
plt.title('single triangle...almost')
plt.axis('equal')
plt.show()
```

Coding concept check: why is there a missing edge in the plot?

```
# Go all the way around!
face = faces[0]
plt.figure()
plt.plot(np.append(x[face], x[face[0]]), np.append(y[face], y[face[0]]), '-o')
plt.title('A Single Triangle')
plt.axis('equal')
plt.show()
```

Coding concept check: do you understand what appending the 0th element does above?

We can also do this with triplot():

```
plt.figure()
plt.triplot(x, y, faces, color='black')
```

```
plt.plot(x, y, 'o', color='blue')  # plot the points as dots
plt.title('A Single Triangle')
plt.axis('equal')
plt.show()
```

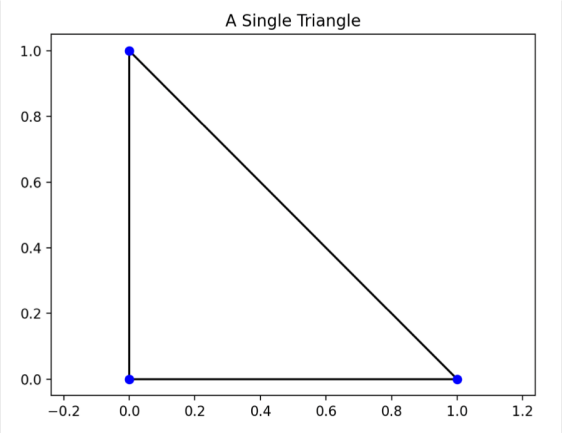

Figure 5: A single triangle plotted using ***triplot****.*

2c. A Triangulated Square

We build a square from two triangles.

```
x = np.array([0, 1, 0, 1])      # 4 corner points of a square
y = np.array([0, 0, 1, 1])
faces = [[0, 1, 2], [1, 3, 2]]  # two triangles

plt.figure()
plt.triplot(x, y, faces, color='black')
plt.plot(x, y, 'o', color='blue')
plt.title('2D Triangulation of a Square')
plt.axis('equal')
plt.show()
```

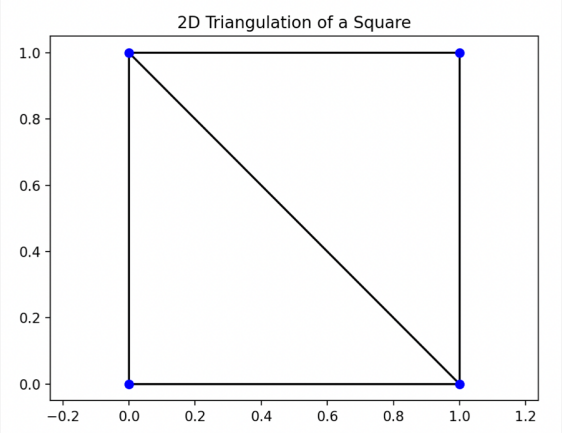

Figure 6: A square composed of two triangles.

2d. A Triangulated Cube in 3D

Now we move to 3D, building a cube from 8 corner points and 12 triangles. Can you build this yourself?

```
vertices = np.array([
    [0, 0, 0],  # 0
    [1, 0, 0],  # 1
    [1, 1, 0],  # 2
    [0, 1, 0],  # 3
    [0, 0, 1],  # 4
    [1, 0, 1],  # 5
    [1, 1, 1],  # 6
    [0, 1, 1]   # 7
])

# Define faces (two per face, 12 faces total)
faces = [
    [0, 2, 1], [0, 3, 2],  # bottom face
    [4, 5, 6], [4, 6, 7],  # top face
    [0, 1, 5], [0, 5, 4],  # front face
    [2, 3, 7], [2, 7, 6],  # back face
    [0, 7, 3], [0, 4, 7],  # left face
    ?????? <Fill in here>   # right face
]


fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
tri_faces = [[vertices[i] for i in tri] for tri in faces]  # list of 3D triangles
ax.add_collection3d(Poly3DCollection(tri_faces, facecolors='cyan', edgecolors='black', alpha=0.8))
ax.scatter(vertices[:,0], vertices[:,1], vertices[:,2], color='blue')

# Set labels
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')

# Set equal aspect ratio
ax.set_box_aspect([1, 1, 1])
plt.title('3D Triangulation of a Cube')
plt.show()
```

Try moving it around.

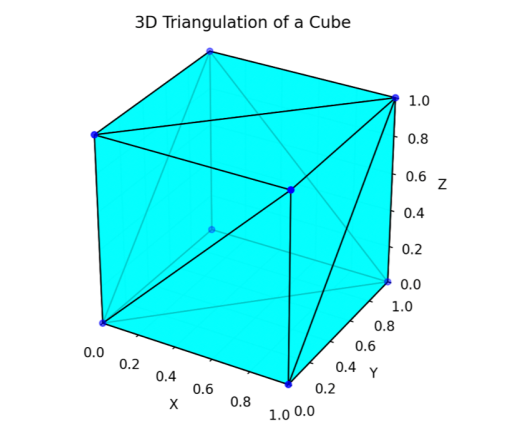

Figure 7: Triangulated cube from 8 vertices and 12 triangles.

3. Mesh Validation

Before doing analysis or alignment, we often must ensure that the mesh is well-formed: - **Watertightness**: no missing connections or open edges. - **Consistent Orientation**: all face normals point outward.

3a. Watertightness Check

Let's count how many times each edge appears. If watertight, it should appear twice. This demo highlights the use of dictionaries.

```
# Initialize an empty dictionary to count edges
edge_count = {}

# Loop over all faces
for tri in faces:
    # Extract edges from the triangle
    edges = [
        (min(tri[0], tri[1]), max(tri[0], tri[1])),
        (min(tri[1], tri[2]), max(tri[1], tri[2])),
        (min(tri[2], tri[0]), max(tri[2], tri[0]))
    ]

    # Count how many times each edge appears
    for edge in edges:
        if edge in edge_count:
            edge_count[edge] += 1
        else:
            edge_count[edge] = 1

# Collect edges that appear < 2 or > 2 times (open or non-manifold edges)
open_edges = []
for edge in edge_count:
    if edge_count[edge] != 2:
        open_edges.append((edge, edge_count[edge]))

# Report
if len(open_edges) == 0:
    print("The mesh is closed (watertight).")
else:
    print("The mesh is NOT closed. Problematic edges:")
    for edge, count in open_edges:
        print(f"  Edge {edge} appears {count} times")
```

Concept check: why did we use min() and max() in the definition of edges?

3b. Face Orientation (somewhat advanced)

Recall that the order of a triangle matters: the order of vertex indices listed for a triangle determines which way is "in" or "out" in 3D. Here we do a simple check for whether the faces of the cube are pointing the right way. Let's use dot products to check if face normals point away from the mesh center. A dot product measures the amount of alignment between two vectors. If the face's "normal" direction (ie its "outward" direction) is pointing in a similar direction as the vector pointing from the origin to the face, then the dot product will be positive. If the face is oriented the wrong way, then the dot product is negative.

```python
# Compute cube center
center = np.mean(vertices, axis=0)

def is_outward_facing(tri):
    v0, v1, v2 = vertices[tri[0]], vertices[tri[1]], vertices[tri[2]]
    # Compute normal vector
    normal = np.cross(v1 - v0, v2 - v0)
    # Vector from the center of the triangle to center of the cube
    tri_center = (v0 + v1 + v2) / 3
    from_center = tri_center - center
    # Dot product tells us if the normal is pointing toward or away from the center
    return np.dot(normal, from_center) > 0  # Should be negative if pointing outward

# Check all faces
inward_facing = []
for i, tri in enumerate(faces):
    if not is_outward_facing(tri):
        inward_facing.append(i)

# Report
if len(inward_facing) == 0:
    print("All faces are consistently outward-facing.")
else:
    print("Found inward-facing faces at indices:")
    print(inward_facing)
```

Concept check: This works for a cube. Would it work for a more complex shape (ex the Stanford bunny)? Provide a simple example that fails, and verify this with code.

3c. Advanced checks

Challenge: how does this check work? At the least, make sure you can follow the structure: we use the `Mesh` class imported from mesh.py. The class has a *method* called `is_closed`. A method is like a function that belongs to a class. The class instance `mm` executes the method `is_closed()` when we say `mm.is_closed()`, effectively asking itself, 'Am I closed?'

```
mm = Mesh(vertices, faces)
mm.is_closed()

# To fix face orientations using Mesh() class, do the following:
mm.make_normals()
mm.force_z_normal(direction=1)
```

---

4. Loading meshes from microscopy data

We can now load some experimentally-derived 3D triangle meshes — for example, segmentations from a microscopy dataset of the *Drosophila* gut. We've saved the mesh as a `.ply` file. Take a look at the PLY file in a text editor. Note that all the information about the contents are in the header. First the vertices are listed, then the faces. The header declares what information is in the file, how many vertices, how many faces, etc.

Below we use the `pyvista.PolyData` class to provide convenient 3D plotting for meshes. You can use the more standard `matplotlib.pyplot` functions too, but they are going to be laggy for meshes of this size (or if you don't have enough RAM, they will make your python console freeze up).

```
m = pv.read('./wt/20240527/mesh_000000_APDV_um.ply')  # returns a PyVista mesh object
m.plot(show_edges=True)
```

Figure 8: Example mesh from level sets and Poisson disk surface reconstruction.

---

5. Shape Matching Using ICP

The **Iterative Closest Point (ICP)** algorithm finds the rigid transform (rotation + translation) that best aligns two point clouds or meshes.

5a. Generate and Offset Meshes

Let's load a couple of meshes – two guts from different embryos at a similar developmental stages. We load these as `TriangleMesh` class instances from Open3D, but don't let that scare you: they are just containers for vertices and faces. We then compute the optimal translation and rotation that best matches one mesh to the other one. I packaged this optimization in a function in `organ_geometry.py`.

```
from organ_geometry import align_mesh_icp, color_mesh_by_distance

ssfactor = 10 # subsampling factor (take 1/N points for the ICP registration).
              # Larger values will run faster but be less precise. This is for speedup

print('Reading in meshes...')
fA = "wildtype/20240527/mesh_000040_APDV_um.ply"
meshA = pv.read(fA)

fB = "wildtype/20240531/mesh_000050_APDV_um.ply"
meshB = pv.read(fB)

# Align one mesh to the other
meshA_t, T = align_mesh_icp(meshA, meshB, ssfactor=ssfactor)
```

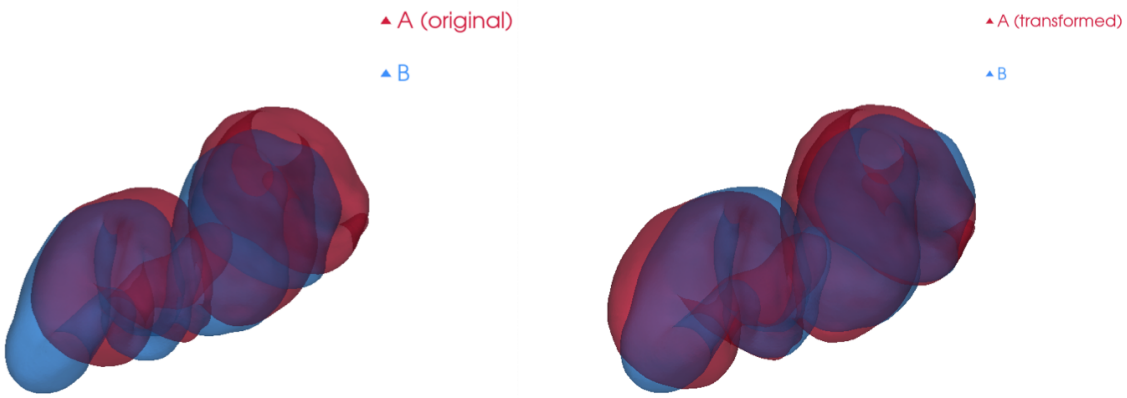

Figure 9: Two different embryos' midguts, captured at a similar stage of development, aligned in space.

Here `T` is a 4×4 NumPy array representing the optimal rigid transform. Advanced concept check: how are data stored inside `T`? Hint: There is a matrix encoding rotations and scaling, but also a translation (dx, dy, dz). Which columns/rows are which?

5c. Compute and Color Distance

We measure how close the aligned mesh is to the target and color it by the distance from the nearest matching point.

```
# Color the mesh by distance between the two
color_mesh_by_distance(meshA_t, meshB, transform=None)
```

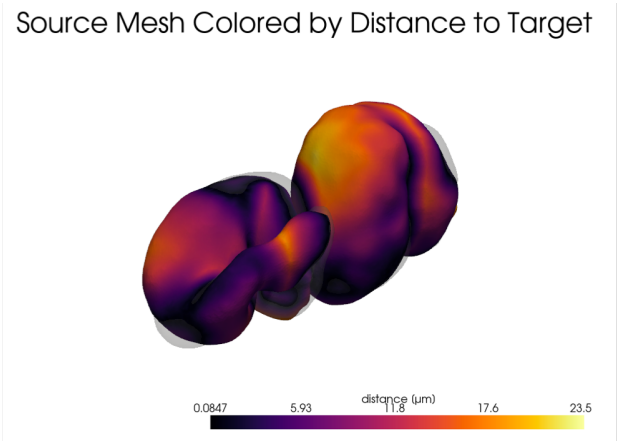

Figure 10: The Euclidean distance between two gut meshes.

Under the hood, in another function I put in `organ_geometry.py`, we computed the distance to the nearest meshB vertex for each vertex in meshA. Then we converted those distances to a color array, a NumPy array with shape `(N, 3)` for RGB values. How does your alignment look?

5d. Generate and Offset Meshes

The guts we've been looking at are from a wildtype embryo with three different fluorescent markers (full genotype is *w, HandGFP / w; UAS-mCh:CAAX / +;bynGAL4, klar / H2A-iRFP*. Let's compare to the gut of an embryo that overexpresses Myosin1C (full genotype is *w, HandGFP / w; UAS-Myo1C:RFP / +;bynGAL4, klar / H2A-iRFP*). In this case, the unconventional myosin motor acts in a portion of the digestive tract (the *byn* domain) to invert the chiral shape dynamics.

How similar are these two organ morphologies?

Let's first compare them directly.

```
print('Reading in meshes...')
fA = "wildtype/20240527/mesh_000050_APDV_um.ply"
meshA = pv.read(fA)

fB = "bynGAL4_UASMyo1C/20240528/mesh_000052_APDV_um.ply"
meshB = pv.read(fB)

# Align one mesh to the other
meshA_t, T = align_mesh_icp(meshA, meshB, ssfactor=ssfactor, Alabel="WT", Blabel='byn>Myo1C')

# Color the mesh by distance between the two
dists, plotter = color_mesh_by_distance(meshA_t, meshB, transform=None)
plotter.show()
```

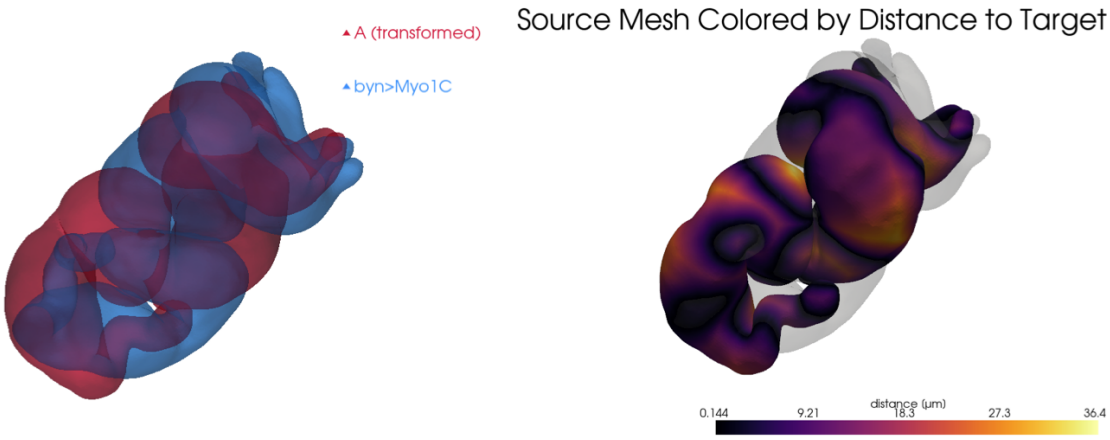

Figure 11: Overexpression of Myo1C dramatically changes the gut shape.

Now let's flip one across the left-right axis and compare them again.

```
# Now flip y -> -y in the Myo1C OE case
meshA = pv.read(fA)
meshB = pv.read(fB)
meshB.points[:, 1] *= -1

# Align one mesh to the other
meshA_t, T = align_mesh_icp(meshA, meshB, ssfactor=ssfactor, Alabel="WT", Blabel='byn>Myo1C, mirrored')

# Color the mesh by distance between the two
dists, plotter = color_mesh_by_distance(meshA_t, meshB, transform=None)
plotter.show()
```

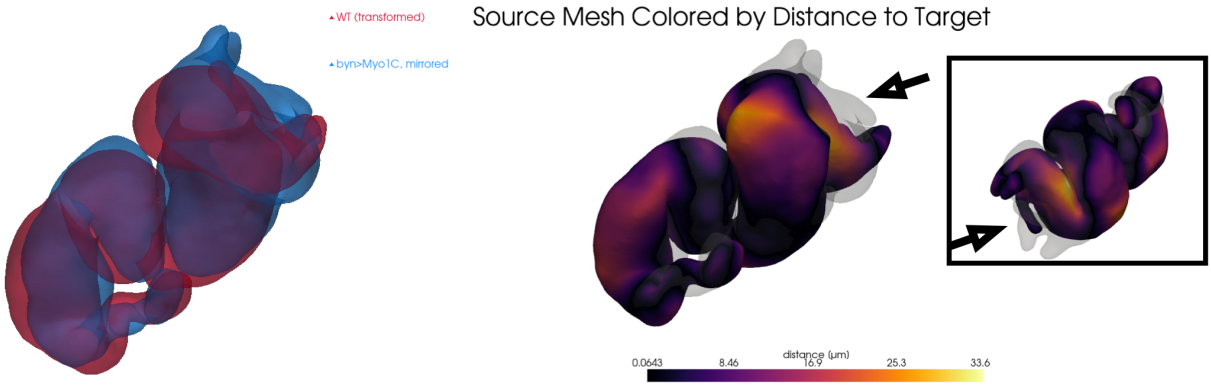

Figure 12: Overexpression of Myo1C nearly mirrors (inverts along the left-right axis) the gut shape. Pronounced residual mismatch is visible in the anterior chamber (arrowhead).

Part 2: Timepoint Alignment and Spatiotemporal Comparison

This second part builds on our introductory ICP module to analyze time-series data from a developing embryonic gut in WT conditions and with Myo1C overexpression. We align shapes over time, compare shape dynamics between experimental conditions, and visualize mismatch using point cloud alignment. This module assumes that triangulated mesh files (`.ply`) are already segmented from microscopy and stored in directories corresponding to timepoints.

---

6. Setup and Parameters

We begin by importing libraries and setting parameters. These include which experimental conditions we want to compare (`wt`, `oe`, etc.) and how much to subsample meshes for faster computation.

```
from organ_geometry import *
import pandas as pd

# Main parameters
step = 3            # downsampling step in units of timepoints
                    # (higher will run faster and consider fewer timepoints)
ssfactor = 10       # spatial subsampling factor for ICP
                    # (higher is faster but potentially less accurate)
wt_oe = 'wt'        # comparison will initially be between two WT datasets
outdir = 'results' # where to store output on disk
dt = 2              # timestep in minutes between adjacent timepoints (this should not be changed, taken

if not os.path.exists(outdir):
    os.mkdir(outdir)

dirA = "wildtype/20240527/"
dirB = "wildtype/20240531/"
flipy = False
```

---

7. Timepoint Matching with ICP

This section performs pairwise shape comparison using ICP to align all meshes from `dirA` to those in `dirB`. ICP produces a matrix of matching errors, which we smooth and use to infer a time-mapping between the two datasets.

```
# Collect all PLY files and extract their timepoint numbers
# List and sort .ply files
filesA = []
for f in os.listdir(dirA):
    if f.endswith(".ply"):
        filesA.append(f)

filesA = natsorted(filesA)[::step]

# Do the same for dirB
filesB = []
for f in os.listdir(dirB):
    if f.endswith(".ply"):
        filesB.append(f)

filesB = natsorted(filesB)[::step]

# Extract timepoints from filenames
tpsA = np.array([extract_tp(f) for f in filesA])
tpsB = np.array([extract_tp(f) for f in filesB])

# Compute ICP RMSE
icp_raw = build_icp_cost_matrix(dirA, dirB, ssfactor=ssfactor, step=step, flipy=flipy)
icp_smooth = smooth_icp_matrix(icp_raw)
```

Now plot the result.

```
ig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), sharey=True)
im1 = ax1.imshow(icp_raw, cmap='inferno')
ax1.set_title("Timeline comparison via ICP")
ax1.set_xlabel("Time B")
ax1.set_ylabel("Time A")
fig.colorbar(im1, ax=ax1, label="ICP Error")

im2 = ax2.imshow(icp_smooth, cmap='inferno')
ax2.set_title("Timeline comparison via ICP")
ax2.set_xlabel("Time B")
ax2.set_ylabel("Time A")
fig.colorbar(im2, ax=ax2, label="Smoothed ICP Error")

plt.show()
```

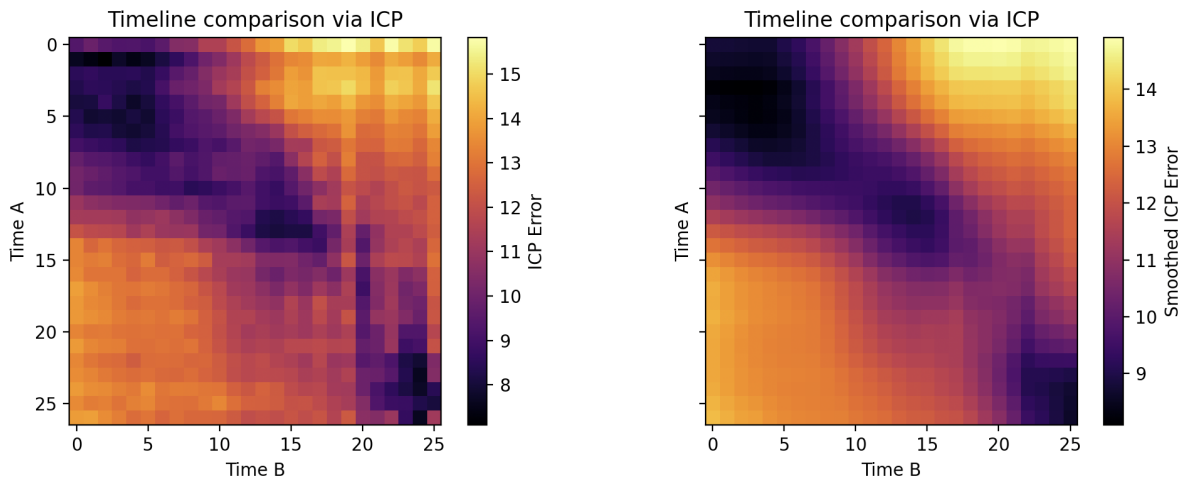

Figure 13: Raw and smoothed root-mean-squared error measurements after ICP alignment for two different WT datasets.

Match timepoints in this matrix by finding row and column minima.

```
# Match timepoints
AtoB, BtoA = match_timepoints(icp_smooth)

# Advanced: using dynamic time warping, we optimize with monotonicity
[path, _, _] = dtw_match(icp_smooth)
path = np.array(path)
path_tps = np.array([tpsA[path[:, 0]], tpsB[path[:, 1]]]).T
```

---

8. Visualizing Timepoint Matching

Plot the smoothed ICP error matrix, with the dynamic time warping path overlayed in red. This helps visualize which timepoints from A match which timepoints from B.

```
plt.figure()
plt.imshow(icp_smooth, cmap='inferno')
plt.plot(path[:,1], path[:, 0], '.-', lw=2, color='tab:purple')
plt.plot(AtoB, np.arange(len(tpsA)), '.-', lw=2, color='tab:blue')
plt.plot(np.arange(len(tpsB)), BtoA, '.-', lw=2, color='tab:orange')
plt.title("Timepoint Matching")
plt.xlabel("Time B")
plt.ylabel("Time A")
plt.colorbar(label="ICP Error")
plt.axis('equal')
plt.tight_layout()
plt.show()
```

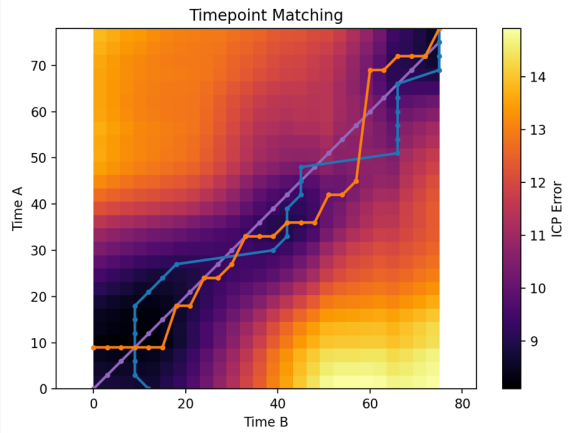

Figure 14: The paths represent AtoB alignment (blue), BtoA alignment (orange), and the result of a simple dynamic time warping implementation (purple).

```
# Plot the result
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

# === Subplot 1: A→B ===
ax1.plot(tpsA, icp_raw[np.arange(len(tpsA)), AtoB], 'o--', label='A→B raw', color='tab:blue')
ax1.plot(tpsA, icp_smooth[np.arange(len(tpsA)), AtoB], 'o-', label='A→B smoothed', color='tab:blue')
# ax1.plot(path_tps[:, 0], icp_smooth[path[:, 0], path[:, 1]], '*-', label='DTW smoothed (A→B)', color=
ax1.set_xlabel("Timepoint A")
ax1.set_ylabel("ICP Error")
ax1.set_title("A→B Matching")
ax1.grid(True)
ax1.legend()

# === Subplot 2: B→A ===
ax2.plot(tpsB, icp_raw[BtoA, np.arange(len(tpsB))], 'o--', label='B→A raw', color='tab:orange')
ax2.plot(tpsB, icp_smooth[BtoA, np.arange(len(tpsB))], 'o-', label='B→A smoothed', color='tab:orange')
# ax2.plot(path_tps[:, 1], icp_smooth[path[:, 0], path[:, 1]], '*-', label='DTW smoothed (B→A)', color=
ax2.set_xlabel("Timepoint B")
ax2.set_title("B→A Matching")
ax2.grid(True)
ax2.legend()
plt.savefig(os.path.join(outdir, f'TPMatching_AtoB_BtoA_{wt_oe}_step{step}_ss{ssfactor}.png'))
plt.show()
```

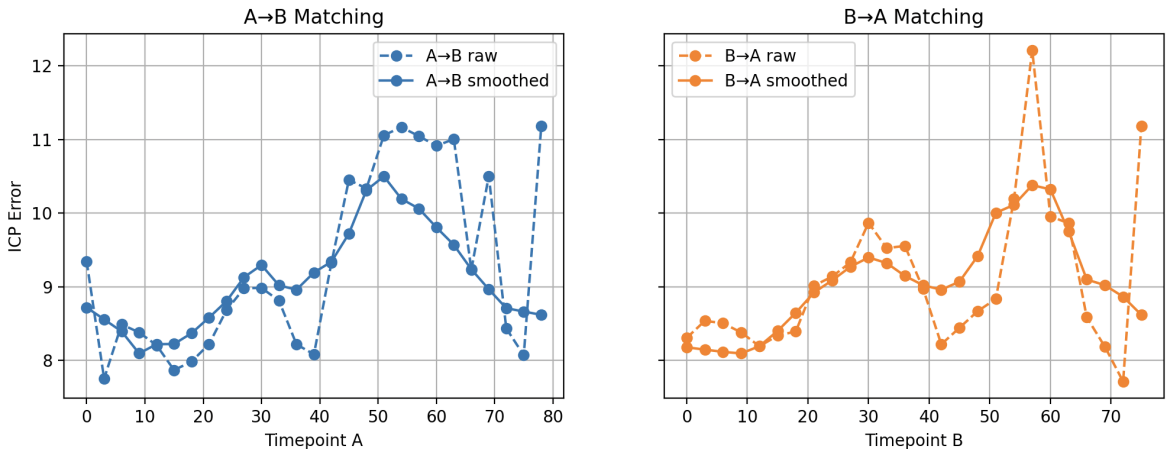

Figure 15: The average root-mean-squared mismatch at each timepoint along the AtoB and BtoA paths show that cross-shape variation rises slightly over time.

---

9. Expected ICP Error from Subsampling

We estimate the smallest error we can expect from subsampling, based on average spacing between points. This gives a baseline to interpret ICP error values.

```
mean_spacing_A = []
mean_spacing_B = []

for fA in filesA:
    # Load and subsample point cloud
    vA = pv.read(os.path.join(dirA, fA)).points[::ssfactor]
    # Compute and store mean spacing
    mean_spacing_A.append(mean_point_spacing(vA))

for fB in filesB:
    # Load and subsample point cloud
    vB = pv.read(os.path.join(dirB, fB)).points[::ssfactor]
    # Compute and store mean spacing
    mean_spacing_B.append(mean_point_spacing(vB))

mean_spacing_A = np.array(mean_spacing_A)
mean_spacing_B = np.array(mean_spacing_B)
expected_rmse_A = mean_spacing_A / np.sqrt(2)
expected_rmse_B = mean_spacing_B / np.sqrt(2)
```

Now let's plot what this looks like.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4), sharey=True)

# === Subplot 1: A→B ===
ax1.plot(tpsA, icp_raw[np.arange(len(tpsA)), AtoB], 'o--', label='A→B raw', color='tab:blue')
ax1.plot(tpsA, icp_smooth[np.arange(len(tpsA)), AtoB], 'o-', label='A→B smoothed', color='tab:blue')
ax1.plot(tpsA, expected_rmse_A, '.--', label='B→A baseline', color='tab:blue')
# ax1.plot(path_tps[:, 0], icp_smooth[path[:, 0], path[:, 1]], '*-', label='DTW smoothed (A→B)', color=
ax1.set_xlabel("Timepoint A")
ax1.set_ylabel("ICP Error")
```

```
ax1.set_title("A→B Matching")
ax1.grid(True)
ax1.legend()

# === Subplot 2: B→A ===
ax2.plot(tpsB, icp_raw[BtoA, np.arange(len(tpsB))], 'o--', label='B→A raw', color='tab:orange')
ax2.plot(tpsB, icp_smooth[BtoA, np.arange(len(tpsB))], 'o-', label='B→A smoothed', color='tab:orange')
ax2.plot(tpsB, expected_rmse_B, '.--', label='A→B baseline', color='tab:orange')
# ax2.plot(path_tps[:, 1], icp_smooth[path[:, 0], path[:, 1]], '*-', label='DTW smoothed (B→A)', color=
ax2.set_xlabel("Timepoint B")
ax2.set_title("B→A Matching")
ax2.grid(True)
ax2.legend()
plt.savefig(os.path.join(outdir, f'TPMatching_AtoB_BtoA_expectedRMSE_{wt_oe}_step{step}_ss{ssfactor}.png
plt.show()
```

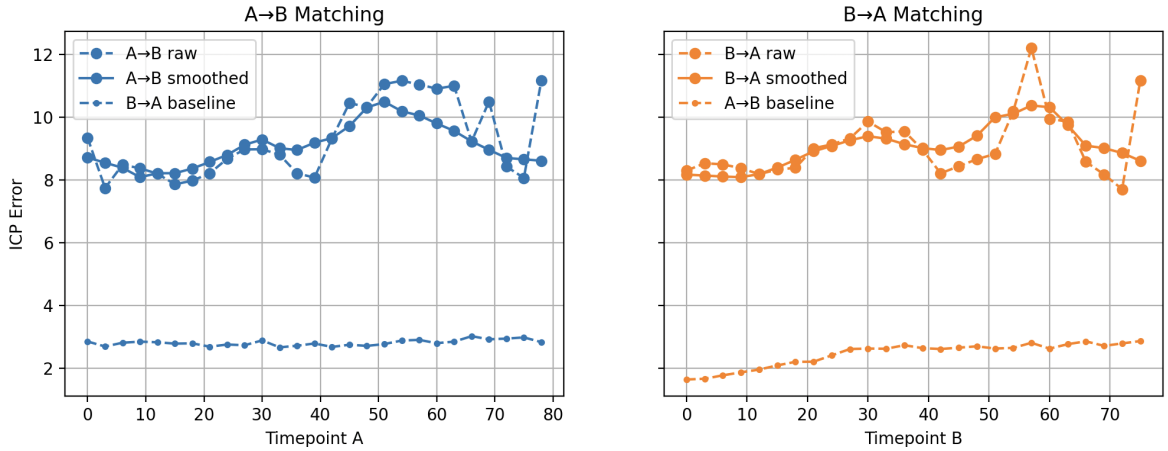

Figure 16: The expected baseline ICP error from finite sampling of the meshes lies far below the measured values.

---

10. Visualize Aligned Meshes

This section loads each matching pair of meshes, re-runs ICP alignment, and overlays them to assess alignment quality. We apply a pre-alignment translation to center the shapes, which improves optimization.

```
# ------------------------------------
# Batch overlays
# ------------------------------------
xyzlim = compute_global_bounds(dirA, dirB, filesA, filesB)
batch_icp_overlay(dirA, dirB, filesA, filesB, AtoB,
                  outdir=os.path.join(outdir, f"icp_overlay_{wt_oe}_step{step}_ss{ssfactor}"),
                  ssfactor=ssfactor, xyzlim=xyzlim, flipy=flipy,
                  Alabel=dirA, Blabel=dirB)
```

---

11. Measuring the spatial pattern of misalignment between samples

We loop over all timepoints and visualize distance between each aligned mesh pair. The color intensity shows how well two shapes match locally. The output is saved to the `results` directory.

```
# ------------------------------------
# Batch color the meshes by mismatch distance
# ------------------------------------
clims = (0, 20)
batch_color_by_distance(dirA, dirB, filesA, filesB, AtoB,
                        outdir=os.path.join(outdir, f"colored_distance__{wt_oe}_step{step}_ss{ssfactor}
                        ssfactor=ssfactor, xyzlim=xyzlim,
                        clim=clims, flipy=flipy)
```

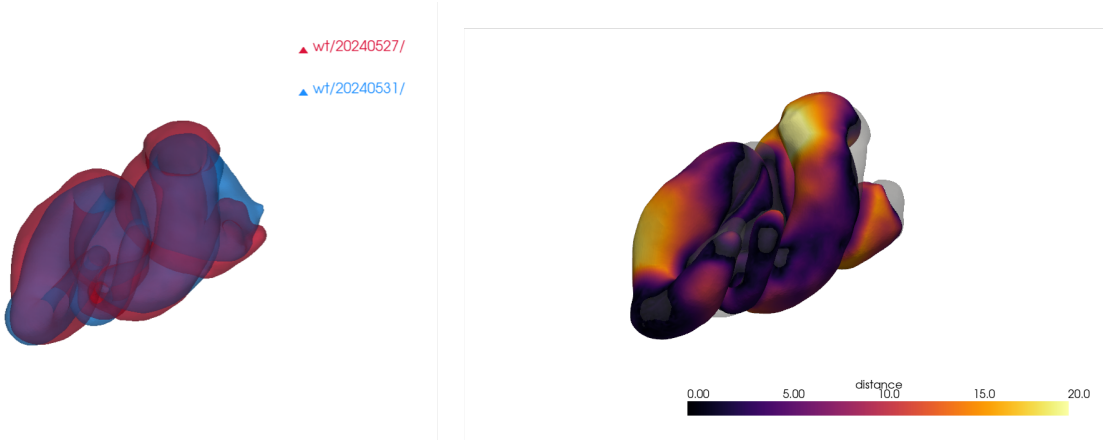

Figure 17: Spatial alignment of two WT midguts at an example timepoint.

---

12. Advanced: PCA-based smoothing as an alternative to dynamic time warping

Dynamic time warping (DTW) gives a monotone, potentially jagged A-to-B mapping (AtoB, BtoA), but sometimes lies "outside" of either set (AtoB or BtoA). Here we instead takes the matched timepoints between two series and smooths their relationship by projecting them into PCA space, smoothing along the orthogonal component, and map back to reconstruct a smoothed trajectory incorporating contributions from both AtoB and BtoA.

```
# ------------------------------------
# Advanced: an alternative to DTW is using both AtoB and
# BtoA to create consensus in PCA1 space.
# ------------------------------------
smoothed_curve = pca_smooth_correspondences(tpsA, tpsB, AtoB, BtoA)
```

We can view the results as usual.

```
plt.figure()
plt.imshow(icp_smooth, cmap='inferno', extent=[tpsB[0], tpsB[-1], tpsA[0], tpsA[-1]],
           origin='lower', aspect='auto')
plt.plot(path[:,1], path[:, 0], '.-', lw=2, color='tab:purple')
plt.plot(tpsB[AtoB], tpsA, '.-', lw=2, color='tab:blue')
plt.plot(tpsB, tpsA[BtoA], '.-', lw=2, color='tab:orange')
plt.plot(smoothed_curve[:, 1], smoothed_curve[:, 0], 'r.-', lw=2)
plt.title("PCA-Smoothed Timepoint Matching")
plt.xlabel("Time B")
plt.ylabel("Time A")
plt.colorbar(label="ICP Error")
plt.axis('equal')
plt.tight_layout()
```

```
plt.savefig(os.path.join(outdir, f'TPMatching_PCASm_{wt_oe}_step{step}_ss{ssfactor}.png'))
plt.show()
```

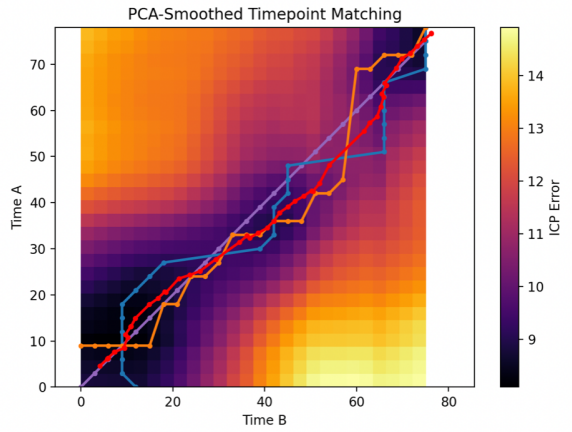

Figure 18: PCA-based smoothing (red curve) offers an alternative for reconciling the AtoB and BtoA timeline matching differences.

---

13. Final Export

We can use a **pandas** dataFrame for saving the result. This is like a table and we save the data as a csv.

```
# Save the results
icpRawA = icp_raw[np.arange(len(tpsA)), AtoB]
icpSmoothA = icp_smooth[np.arange(len(tpsA)), AtoB]
icpRawB = icp_raw[BtoA, np.arange(len(tpsB))]
icpSmoothB = icp_smooth[BtoA, np.arange(len(tpsB))]
df_icp = pd.DataFrame({
    "tpsA": tpsA,
    "AtoB": AtoB,
    "icpRawA": icpRawA,
    "icpSmoothA": icpSmoothA
})

df_icp_B = pd.DataFrame({
    "tpsB": tpsB,
    "BtoA": BtoA,
    "icpRawB": icpRawB,
    "icpSmoothB": icpSmoothB
})

df_icp.to_csv(f"icp_error_AtoB_{wt_oe}_step{step}_ss{ssfactor}.csv", index=False)
df_icp_B.to_csv(f"icp_error_BtoA_{wt_oe}_step{step}_ss{ssfactor}.csv", index=False)
```

You can save numeric results as `.npy` arrays for further analysis. These files store ICP matrices, expected noise floors, and timepoint mappings.

```
np.save(os.path.join(outdir, f"icp_raw_{wt_oe}_step{step}_ss{ssfactor}.npy"), icp_raw)
np.save(os.path.join(outdir, f"icp_smooth_{wt_oe}_step{step}_ss{ssfactor}.npy"), icp_smooth)
np.save(os.path.join(outdir, f"expected_rmse_A_{wt_oe}_step{step}_ss{ssfactor}.npy"), expected_rmse_A)
np.save(os.path.join(outdir, f"expected_rmse_B_{wt_oe}_step{step}_ss{ssfactor}.npy"), expected_rmse_B)
np.save(os.path.join(outdir, f"path_{wt_oe}_step{step}_ss{ssfactor}.npy"), [path, path_tps])
```

14. Cross-Condition Comparison

Run the above for different conditions `'wt', 'oe', 'x1', 'x2', 'x3', 'x4'`. We compare how ICP errors change across conditions. This can reveal if shape dynamics differ significantly between WT and perturbed embryos.

Note that whitespace matters in python. Just as delimiters like spaces or punctuation in English can completely change the meaning of a sentence — for example, "Let's eat, Grandma" vs. "Let's eat Grandma" — Python relies on whitespace, punctuation, and syntax to interpret code correctly.

```
for i, wt_oe in enumerate(conditions):
    if wt_oe == 'wt':
        # dirA = "HandGFPbynGAL4klar_UASmChCAAXHiFP/20240527/"
        # dirB = "HandGFPbynGAL4klar_UASmChCAAXHiFP/20240531/"
        dirA = "wt/20240527/"
        dirB = "wt/20240531/"
        flipy = False
    elif wt_oe == 'oe':
        dirA = "oe/20240528/"
        dirB = "oe/20240626/"
        flipy = False
    elif wt_oe == 'x1':
        dirA = "wt/20240527/"
        dirB = "oe/20240528/"
        flipy = True
    elif wt_oe == 'x2':
        dirA = "wt/20240527/"
        dirB = "oe/20240626/"
        flipy = True
    elif wt_oe == 'x3':
        dirA = "wt/20240531/"
        dirB = "oe/20240528/"
        flipy = True
    elif wt_oe == 'x4':
        dirA = "wt/20240531/"
        dirB = "oe/20240626/"
        flipy = True
    ...
    # All the batch analysis you ran above here
    ...


# ------------------------------------------------------------
# Compare difference between WT and OE against
# diff within WT and within OE
# ------------------------------------------------------------

conds = ['wt', 'oe', 'x1', 'x2', 'x3', 'x4']
icp_dict = {}

for cond in conds:
    fname = os.path.join(outdir, f"icp_error_AtoB_{cond}_step{step}_ss{ssfactor}.csv")
```

```
    df = pd.read_csv(fname)
    icp_dict[cond] = {
        "tps": df["tpsA"].values,
        "error": df["icpSmoothA"].values
    }

# Align on common timepoints
common_tps = np.intersect1d(np.intersect1d(np.intersect1d(icp_dict['wt']['tps'], icp_dict['oe']['tps'])
                                           np.intersect1d(icp_dict['x1']['tps'], icp_dict['x2']['tps']))
                            np.intersect1d(icp_dict['x3']['tps'], icp_dict['x4']['tps']))
for key in icp_dict:
    mask = np.isin(icp_dict[key]["tps"], common_tps)
    icp_dict[key]["tps"] = icp_dict[key]["tps"][mask]
    icp_dict[key]["error"] = icp_dict[key]["error"][mask]

# Stack WT vs OE error
wt_err = icp_dict['wt']["error"]
oe_err = icp_dict['oe']["error"]
tps = icp_dict['wt']["tps"]

# Mean between-group difference
diff_wtoe = np.abs(wt_err - oe_err)
mean_diff = np.mean(diff_wtoe)

# Stack X1-X4 errors
x_errors = np.stack([icp_dict[f"x{i}"]["error"] for i in range(1, 5)])
x_mean = np.mean(x_errors, axis=0)
x_std = np.std(x_errors, axis=0)

# Plot
plt.figure(figsize=(10, 5))
plt.plot(tps*dt, wt_err, 'b.-', label="WT-WT")
plt.plot(tps*dt, oe_err, 'r.-', label="OE-OE")
plt.plot(tps*dt, x_mean, 'k-', label="WT-to-OE (mean x1-x4)")
plt.fill_between(tps*dt, x_mean - x_std, x_mean + x_std, color='gray', alpha=0.3, label="WT-OE ± std")
plt.xlabel("reference time [min]")
plt.ylabel("ICP RMSE (smoothed)")
plt.title("Within- vs cross-ensemble ICP RMSE")
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig(os.path.join(outdir, 'cross_ensemble_RMSE.png'))
plt.show()

print(f"Mean WT-OE diff: {mean_diff:.3f}")
print(f"WT internal std: {np.std(wt_err):.3f}")
print(f"OE internal std: {np.std(oe_err):.3f}")
```

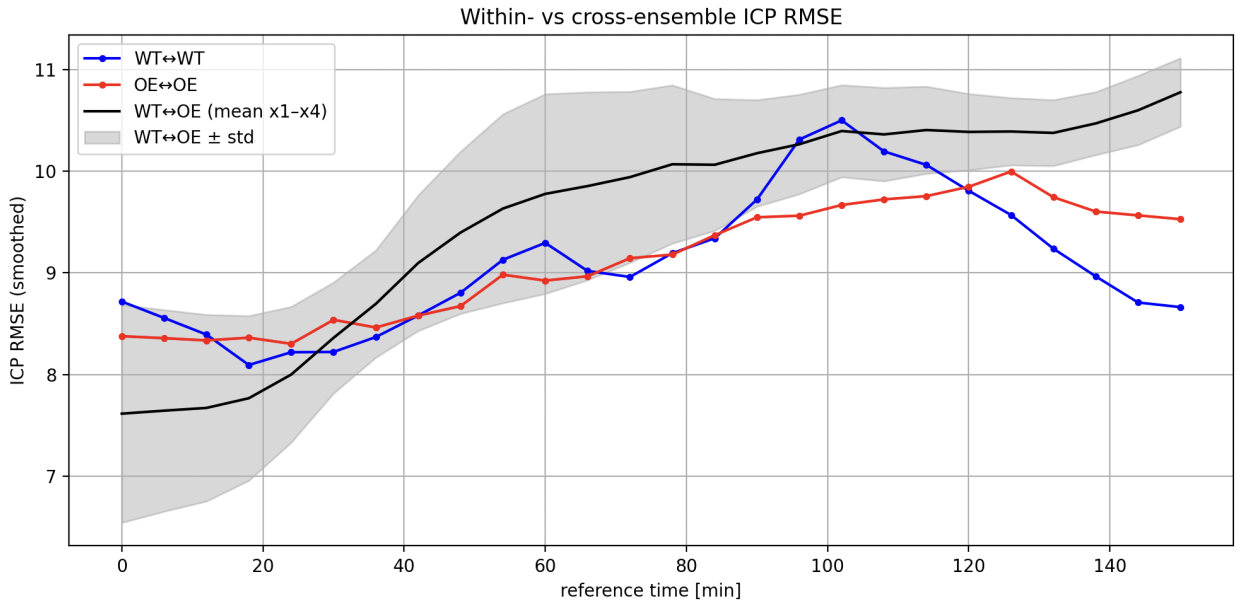

Figure 19: Comparing the distance between mesh pairs within and across groups (with the Myo1C OE case inverted L<->R).

This figure provides a quantitative summary of dynamic shape variability under genetic or experimental perturbation. How do you interpret these results?

Next Steps

- Try aligning real meshes from your experiments
- Explore ICP variants (e.g., point-to-plane)
- Add mesh preprocessing: smoothing, simplification, hole filling
- Analyze biological trends in alignment error